

Configurable synthetic application for studying malleability in HPC

March 9, 2023

1 Introduction

This work presents a synthetic application that allows to configure benchmarks to analyse malleability in applications. It can be configured in order to emulate the behavior that a iterative scientific MPI application could have when it is re-configured during its execution varying the number of allocated processes. This tool allows to study the behaviour of these applications on different scenarios/situations and helps to analyse their performance. The application computational behaviour and different reconfigurations during its execution are described by a set of parameters stored in a file. A execution of this benchmark generates several output files containing different performance measures.

Although producer-consumer applications have been targeted by malleability [1, 2], mainly, it has been widely studied and applied to iterative applications [3-5]. So, we have focused our work to emulate the behaviour of iterative MPI applications that can be resized (shrink or expanded) on-the-fly. Nevertheless, the proper use of the configuration parameters will allow to emulate the behaviour of almost any type of MPI application.

Two definitions related to a resize operation must be considered. A group of processes is considered as *parents* when they spawns to other set regarded as *children* (child group). In some cases, after resizing, parent processes also will belong to the child group. In any case, the whole set of child processes will proceed the execution.

The rest of this manual is organized as follows. Section 2 describes the components of the synthetic application and how it works. Section 3 analyse the parameters of the configuration file that model the application behaviour and its different reconfigurations. Section 4 explains how to install and use this tool. Section 5 describes how to use the analysis tool for the executions and how the final results will be represented. Section 6 show some examples of how to use the application and analyze results.

2 SYNTHETIC MPI MALLEABLE APPLICATION

This section describes how our synthetic application works. It is consisted of five main modules: (i) *Initialization*, (ii) *Application simulation (Computation/-Communication)*, (iii) *Resize*, (iv) *Monitoring* and (v) *Completion*.

Next subsections explain the functionality of these modules and the configuration parameters.

2.1 Initialization

Initialization module is in charge of starting the simulations execution. The main task of this module is carried out by the first group of processes (those starting execution). These ones read the parameters from the configuration file and to broadcast them the rest of groups in each resize as they are needed. Moreover, this module is responsible for initializing the others modules of the synthetic application. Next, the group will start the execution of the simulated application in the *Application simulation* module. This last step is also performed by new processes after a resize. Next times, parent process with rank 0 broadcast the parameters to child processes.

In addition, this module is also in charge of computing some parameters (non-direct) that depend on others defined in the configuration file and on the system where the application is executed. Some non-direct parameters are calculated in each resize because they depend on the configuration of each process group. The remainder of these parameters are calculated only once and sent from parent processes to children in each resize. All of them will be explained in more detail later.

2.2 Application Simulation

This tool is able to simulate iterative applications where an iteration is composed of a sequence of stages of computation and/or communication (called processing stages). An iteration is considered as completed when all these stages have ended. Each one of these processing stages can be configured by parameters.

Types of computation stages:

- Compute-bound applications: Simulates a type of application where CPU utilization is the main factor of the computational cost. It computes Monte Carlo method to estimate π and no data memory access is required.

Types of communication stages:

- Point to point: This communication is performed by calling `MPI_Sendrecv`. We have assumed there are Y nodes with X processes in each one. Inside each node, ranks in the first $X/2$ slots communicates with the upper $X/2$ slots of the previous nodes. On the other hand, the upper $X/2$ slots processes of a node communicate with the first $X/2$ slots of the next node.

This ensures that all processes perform inter-communications to reach other processes.

- Collective one-to-all: This is a communication that involves all processes in the group, but only one sends data. It is performed with a single `MPI_Bcast` operation.
- Collective all-to-all: This communication involves all processes in the group, all of them sending and receiving information. It is performed by `MPI_Allgatherv`.
- Reduction: This is a communication that involves all processes in the group, but only one receives data. It is performed by `MPI_Reduction`. During the communication, some computation is executed on the data. This communication can be also performed with `MPI_Allreduce` to allow an all to all alternative.

2.3 Resize

This module allows to resize the application by enabling malleability. A reconfiguration is performed after a fixed number of iterations has been run (Listing 1).

The resize process is done in three steps:

- Communicate with the RMS to show the application availability in being resized.
- Create a new set/group of MPI processes.
- Redistribute data from the parent processes to the new ones (children).

In the current version of the tool, the first step is simulated by parameters in the configuration file indicating if application can be expanded or reduced, assuming more or less resources available in the system, respectively.

Next step, processes are created by using MPI functions that fall in the MPI standard. In [] multiple methods/strategies are described and any of them can be used. The main methods are:

- Baseline method. It resizes from X processes to Y processes. Y processes are created and X processes are removed using `MPI_spawn` function.
- Merge method. This method reduces the amount of spawned processes. To resize from X processes to Y processes, $Y - X$ processes are created when expanding ($X > Y$) and $X - Y$ are removed when shrinking ($X < Y$), introducing new communicators if it is necessary, or reusing and merging existing communicators in other cases.

These methods can be used with any combination of the following strategies:

- Spawning processes asynchronously or synchronously. In the first case, parent processes continue computing iterations while the resize is being performed by a or various additional threads. In the other case, the parents stop their execution.
- Determining the number of parents involved in the spawn operation: only one (rank 0), or all collectively.

Therefore, each method has up to 4 different ways of spawning processes as they can be used with no strategies, one of them, or both.

Also, in order to analyze the impact of the assignment of processes to physical cores, two additional possibilities have been considered:

- Spread: Allocate processes to cores using as many nodes available as possible.
- Compact: Allocate processes to the cores of a node until completed before allocating to another node.

Finally, the last step, data is distributed from parents to children. It is always distributed linearly among the process group. This task can be carried out using two different strategies:

- Synchronously: The application execution is stopped while it is performed. The synchronous redistribution is implemented by a single call to `MPI_Alltoallv` which allows to specify how to send data from parents to children.
- Asynchronously: The execution in parent processes continues while the redistribution operation is carried out.

An asynchronous data redistribution is not always possible because they can be modified in each iteration. Thus, the tool implements data redistribution in two steps. First, constant data (those that does not change during a computation iteration) is sent asynchronously (running continues). Then, variable data is sent by synchronous communication (execution is stopped). The application allows you to configure which part of the data is distributed asynchronously. The rest will be done synchronously.

For asynchronous redistribution there are four different possible implementations, three based on MPI asynchronous primitives and an approach based on *threads*.

The ones based on MPI asynchronous primitives are the following:

- Point to point: Redistributes data using non-blocking point to point functions. The redistribution is considered completed for parents when `MPI_Test` returns successfully for all `MPI_Isend` functions.
- Collective all to all (Parents): Redistributes data using the non-blocking `MPI_Ialltoallv` function. The redistribution is considered completed for parents when `MPI_Test` returns successfully for all parents.

The *thread*-based one requires starting the MPI environment so that multiple threads can make simultaneous calls. This strategy requires a call to *MPI_Init_thread* function with parameter *MPI_THREAD_MULTIPLE*. It launches a new thread in each parent that will take care of the data redistribution by means of the function *MPI_Alltoallv*.

2.4 Monitoring

This module monitors the execution and communication times of the different phases/stages of the simulation. When it ends these values are stored in an intermediate output file. *MPI_Wtime* is used to obtain the following required times:

- T_{stages}^X : Time per stage. Time to perform the stage X in a single iteration.
- T_{iter} : Time per iteration. This time is the sum of T_{stages}^Y for Y stages.
- T_{spawn} : Time required for creating new processes.
- T_{spawn_real} : Time from the point the spawning is started to when the application notices it has ended. This time is equal to T_{spawn} if the asynchronous strategy is not used. If it is used, this time is higher than T_{spawn} as the application only checks if the spawn has finished after completing an iteration.
- T_{SR} : Time used for synchronous data redistribution.
- T_{AR} : Time employed for asynchronous data redistribution.
- T_{total} : Total time from start to end of the application execution.
- *Asynch_Iters*: Total iterations that were performed while a background resizing step was being carried out.

2.5 Completion

This module is in charge of ending the execution for a processes group and storing their timings in the output file for further analysis. It will be run if either, a processes group ends its execution by a resize, or if the last processes group has completed the execution.

2.6 General working

Figure 1 shows a workflow diagram of our tool. In the figure are highlighted by colors the different parts of the synthetic application and grouped in the different modules described: *Initialization* green, *Application simulation* yellow, *Resize* purple, *Monitoring* blue and *Completion* red. The first group of processes starts the execution at the *Initialization* module. A process of this group is in charge of reading all the parameters of the configuration file and storing them

in an internal data structure in order to broadcast them to the other processes and groups after each resizing. Next, *Application simulation* module is run. This module is responsible for computing the total iterations programmed for a group of processes, taking into account the application computational behaviour described in the configuration file. The last step of this part consists of checking whether a new resize is defined in the benchmark. If so, execution continues at the *Resize* module where two tasks are carried out: process spawning and data redistribution from the parent group to the children. Thus, the new group will continue the application execution from *Initialization* module that leads them to run again *Application simulation* module. Thus, the execution computes the total iterations defined for this new group and from the last one on the new resources and considering the computational behavior parameters set for this group. On the other hand, parents group (or part of them) ends their execution in the *Completion* module. This one is responsible to store the application performance monitored by the *Monitoring* module in an intermediate file.

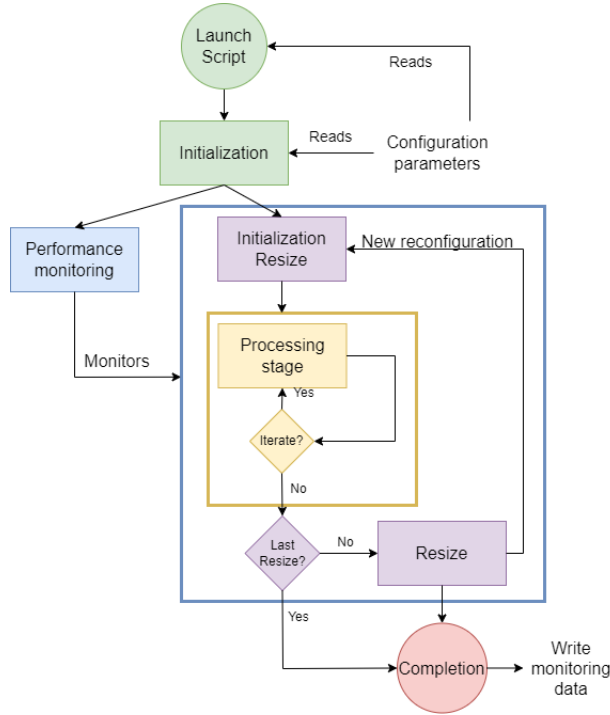


Figure 1: Flowchart of the synthetic application.

Listing 1 showcases an example of a code with all the previously enumerated modules differentiated and separated also by the colors. In this regard, lines 4-12 initialize the application and processes groups; lines 14-22 represent an iterative computation with its different stages of calculation and communications (processing_stages); resizes take place from line 24 to 28; and in line 29 results

```

1 int main() {
2   // First group initializes the application
3   if(init) {
4     if (rank == 0)
5       read_config();
6     broadcast_config(); // Send to all ranks in group
7     calculate_non_direct_parameters();
8
9   } else {
10    broadcast_config(); // Receive from parent rank 0
11    calculate_non_direct_parameters();
12    redistribute_data(); // Receive from parents.
13  }
14  // Job computation
15  for(int iter=0; iter < Iters; iter++) {
16    // Computes to consume T_it time
17    for(int i=0; i < n_stages; i++) {
18      processing_stage(stages[i]);
19    }
20  }
21  // Reconfigure if not last resize
22  if (!last_resize) {
23    create_child_processes();
24    broadcast_config(); // From rank 0 to all children
25    redistribute_data(); //send to children
26  }
27  store_performance_data();
28 }

```

Listing 1: Basic skeleton of the synthetic application.

are stored.

3 Configuration

This tool has two main functionalities. On the one hand, it simulates the application behaviour with certain characteristics and, on the other, it allows performing different resizing during its execution simulating a malleable application. A configuration file containing the necessary parameters to define these simulation is described in this section.

The general parameters dedicated to emulate the execution behaviour of an application are the following:

- **Total_Stages:** Total number of different stages (processing stages) that executes a application during each iteration. Any application is broken down into a sequence of computation and/or communication stages. The

sum of all of them will be this value. It will always be equal or greater than 1.

- **Granularity:** Problem size for computation stages.

The following general parameters are dedicated for resizing in malleable application:

- **Total_Resizes:** Number of resizes to perform during application execution. A value of zero indicates that none will be executed. Each resize is composed by a processes group.
- **SDR:** Number of bytes to redistribute in each resize.
- **ADR:** Number of bytes to redistribute asynchronously. Is also possible to indicate percentage of **SDR** that will be distributed asynchronously with a value from 0 to 100. The latter only works when using the script `multipleRuns.sh`.

After, each processing stage in an iteration is characterized by several parameters that define its behavior:

- **Stage_Type:** type of stage to be simulated. Different possibilities:
 - Computation: Monte Carlo method (0) or matrix-vector multiplication (1).
 - Communication: Point to point (2), Collective one-to-all (3), Collective all-to-all (4), reduction operation (5), all-reduction operation (6).
- **Stage_Time:** Time consumed in a stage in seconds.

If **Stage_Type** is defined as communication, the following parameters are also need:

- **Stage_Bytes:** It indicates the total bytes communicated among processes. This parameter is optional.

Next, **Total_Resizes + 1** sets of parameters are defined characterizing each resize. For each group, the related parameters are the following:

- **Iters:** Total iterations to execute before resizing or ending the application.
- **Procs:** Number of processes in the group.
- **Dist:** Strategy applied to emulate the node allocation: "spread" or "compact".
- **Spawn_Method:** Spawn method to use during resizes: Baseline (0) or Merge (1).

- **Spawn.Strategy**: Spawn strategy to use during resize: None (1), Asynchronous (2), Single parent spawn (3), or All strategies (6).
- **Redistribution.Method**: Redistribution method for data redistribution, either using MPI primitives as All-to-all Parents (0), Point to point (1), RMA Lock (2) or RMA LockAll(3).
- **Redistribution.Strategy**: Method strategy for data redistribution. Can be None (1) or use Threads (2).
- **FactorS**: This speedup factor emulates how the performance of computation stages are affected by the number of processes as it happens in real applications. Values higher than 1 increments the stage time, while values below zero lowers the stage time. It can not have negative values. For example, if the application had an ideal speedup, and the resizing went from 2 to 4 processes for a sequential **Stage.Time** of 4 seconds, **FactorS** would be 0.5 and 0.25, respectively. Thus this parameter determines the scalability of the emulated application. This value influences the calculation of non-direct parameters.

The parameters **Spawn.Method**, **Spawn.Strategy**, **Redistribution.Method** and **Redistribution.Strategy** are ignored when launching the simulation execution since the processes are created by the command *mpirun* or *mpiexec*.

Listing 2 shows a configuration file example that simulates a malleable application which has a iteration composed of 3 processing stages and performs only one resize (2 processes groups). Total bytes to redistribute in each resize are 1GB asynchronously. First stage (**[stage0]**) is a computation based on Monte Carlo method with a problem size of 100000 (**Granularity**). Next stages are communications, **[stage1]** is a **MPI.Allreduce** operation of 8 bytes while the last one (**[stage2]**) is an **MPI.Allgatherv** with a **Stage.Time** of 25ms. The only resize (**[resize1]**) is an expansion from 2 to 10 processes using asynchronous data redistribution based on *threads*. The total iterations performed by first group is 1 and the other group does 10 iterations.

3.1 Modeling interconnection network

This tool provide two options to configure communication stage. The first and more simple one is to indicate the number of bytes to be transferred **Stage.Bytes**. In this case, the simulation is performed from the operation selected and this information.

3.2 Parameters computed for modeling computation stages

If **Stage.Type** is defined as a computation stage, other parameters have to be calculated to complete the simulation:

- T_{op} : Time to execute a single computation of selected type in **Stage.Type** and problem size equal to **Granularity**.

```

[general]
Total_Stages=3                # Number of stages
Granularity=100000            # Computation stages problem size
Total_Resizes=1               # Number of resizes
SDR=10000000000 # Total synchronous redistributed bytes
ADR=100 # Total asynchronous redistributed bytes
;end [general]
[stage0]                       # Computation stage
Stage_Type=0                  # Stage type. Perform pi computation
Stages_Bytes=0                # Bytes to communicate
Stage_Time=0.16
# Stage duration, 160ms
;end [stage0]
[stage1]                       # Communication stage
Stage_Type=6                  # Perform Allreduce
Stage_Bytes=8                 # with 8 bytes
Stage_Time=0                  # not used
;end [stage1]
[stage2]                       # Communication stage
Stage_Type=4                  # Perform Allgatherv
Stage_Bytes=0                 # not used
Stage_Time=0.025              # during 25ms
;end [stage2]
[resize0]
Iters=1                       # Total iterations for this group
Procs=2                       # Number of processes in this group
FactorS=0.5                   # Scalability factor
Dist=spread                   # Physical process mapping type
Spawn_Method=0                # Spawn method
Spawn_Strategy=1              # Spawn strategy
Redistribution_Method=3        # Redistribution method
Redistribution_Strategy=1
# Redistribution strategy
;end [resize0]
[resize1]
Iters=10                      # Total iterations for this group
Procs=10                      # Number of processes in this group
FactorS=0.1                   # Scalability factor
Dist=spread                   # Physical process mapping type
Spawn_Method=0                # Spawn method
Spawn_Strategy=1              # Spawn strategy
Redistribution_Method=3        # Redistribution method
Redistribution_Strategy=1
# Redistribution strategy
;end [resize1]

```

Listing 2: Configuration file example.

- **op**: Number of times that the selected computation in **Stage_Type** has to be executed to achieve the scalability defined in **FactorS** when the sequential cost is **T_stage**.
This is defined as $op = T_stage * FactorS / T_{op}$.
This parameter must be recalculated after a resize if *FactorS* takes a different value.

4 Simulation

This section describes the steps to install this tool and execute the synthetic application with a configuration file simulating a malleable application. We also show the output files obtained that allow to analyse its behaviour and performance. First of all, the requirements for installation and use are explained, followed by a description of how the simulations are carried out and measurements are stored.

4.1 Installation

The installation of this tool requires the following dependencies to be taken into account:

- GNU compiler.
- MPICH 3.x.y. or 4.0.3 Nemesis or OFI netmod. Is not possible to use UCX to create processes dynamically with the current versions of MPICH.
- Python3 interpreter. Only needed to perform multiple executions with different configurations with one submission.
- Slurm Workload manager. Tested with Slurm-wlm 19.05.5. It is optional, but if it is not installed, some limitations apply.

Next, you have to install the synthetic application. The files needed to download it can be obtained by cloning from the public repository, which is located at http://lorca.act.uji.es/gitlab/martini/malleability_benchmark.git.

Then, you can install by going to the directory */path/to/malleability_benchmark/Codes* and use **Make** or **Make install_slurm** command. Use the second command to install the application with Slurm support. After installation the following directory tree can be observed:

- *Codes*: Contains the binary and source codes.
- *Exec*: Holds execution scripts for the simulations.
- *Results*: Stores output files obtained from simulation.
- *Analysis*: Includes scripts for analyzing output files.

Also, optional modules for the Python3 interpreter are needed in order to analyse the output files. They are not mandatory and can be installed in a personal computer without needing to reinstall our tool. These are:

- Numpy module.
- Pandas module.
- Seaborn module.
- Scipy module.
- scikit_posthocs module.
- Matplot module.

4.2 Execution

4.2.1 Execution scripts

The directory *Exec* contains two scripts that allow executing the synthetic application from a configuration file and simulate a malleable application.

The first script is called *singleRun.sh* (Listing 3), and requires the following parameters:

1. Configuration file name. This file contains every parameter needed to simulate the application behaviour and its resizing. It is mandatory.
2. Identifier of execution (*ID*). It is an integer number that identifies this simulation. Executions with an identical configuration file should have the same *ID*. This parameter is optional with a default value to 0.
3. Total number of executions. It is an integer number indicating how many times the simulations will be run. A minimum of 5 is recommended to ensure a proper statistical analysis. It is also optional with a default value to 1.
4. Use Extrae. It is an integer to indicate whether to use Extrae (1) or not (0).
5. Path where the output file should be saved. It is optional and has a default value to current directory. Nevertheless, if a path is given by the user but it does not exist, the script fails and the simulation is not run.

An example to execute this script is shown at Listing 3, where the simulation has a configuration file named *configuration.ini*, an *ID* of 5 and it is executed 20 times.

By default, the script *singleRun.sh* allocates the nodes required for the job using Slurm commands. It should be called with the command *bash*.

Instead, if the user wants his own allocation, the script *singleRunCostum.sh* is provided to be called with the command *sbatch* or *srun*. On the other hand,

```
bash singleRun.sh configuration-file.ini ID Repetitions
Use_Extrae Output_path
```

```
bash singleRun.sh configuration.ini 5 20 0
```

Listing 3: Script *singleRun.sh* example of use.

```
bash multipleRuns.sh array-configuration-file.ini OutpathName
```

```
bash multipleRuns.sh configuration_array.ini config
```

Listing 4: Script *multipleRuns.sh* example of use.

if no allocation is needed is should use *bash* command. This script has the same parameters that *singleRun.sh* uses.

Use the script *singleRunCostum.sh* with the bash command if the installation is performed without Slurm support.

The script *multipleRuns.sh* (Listing 4) allows multiple simulations to be run. Different values represented as a comma-separated list can be defined for certain parameters in the configuration file. This script will create as many configuration files as different configurations can be obtained from the original configuration file. As a result, a directory is created and contains the configuration files of all simulations. This script requires the following 2 parameters:

1. Configuration file name. It is mandatory.
2. Common output configuration file names(CON). All resulting configuration files will have as name "CON+i+.ini", where *CON* is the common name, *i* is an increasing index for each file, and ".ini" is the format file.

An example to execute this script is shown at Listing 4.

The parameters in the configuration file that can be set as a list of values are: *SDR*, *ADR*, *Procs*, *FactorS*, *Dist*, *Redistribution_Method*, *Redistribution_Strategy*, *Spawn_Method*, *Spawn_Strategy*. An example is shown in Listing 5 where parameters used as arrays are shown. From this example, 24 different configurations are created.

If an array is used for *Procs*, its related *FactorS* parameter must also hold an array with equal size. This is because each value of *Procs* should have a unique value for *FactorS*.

Lastly, the script

4.2.2 Error check

The script *CheckRun.sh* (Listing 6) in directory *Exec* checks the consistency of the measurements obtained in the simulations and stored in the output files. It is recommended to perform this check before analyzing the results, so those simulations that have not been executed correctly are launched again.

```

[general]
...
ADR=0,100
;end [general]
.....
[resize0]
...
Procs=2,4,8
FactorS=0.5,0.3,0.2
Dist=spread,compact
...
;end [resize0]
[resize1]
...
Procs=10,20
FactorS=0.18,0.1
...
;end [resize1]

```

Listing 5: Configuration file using value list in parameters.

```
bash CheckRun.sh FilePath
```

Listing 6: Script *CheckRun.sh* example of use.

The script expects the following 5 parameters:

1. Maximum index of the runs.
2. Amount of repetitions per index/run.
3. Total stages in all runs.
4. Total groups of processes in all runs.
5. Maximum valid iteration time across all runs. If an iteration time is higher, that particular repetition inside the run is cleaned and launched again.

4.2.3 Intermediate output files

A simulation generates two types of output files where the measurements are stored. One of them is a global file containing general information of the simulations. The remainder files are related to resizes (one for each processes group in which the application is to be resized). The files follow the following name format:

- `RX_Global.out` for global file.

- $RX_GYNPZ.out$. For resize files.

Where the variables mean:

- X : ID of the execution. Selected when the execution script is launched.
- Y : It is the identification of a group of processes in a resize. For a given simulation, it starts at 0 and for each new group is incremented by 1.
- Z : Number of processes for the group Y .

The names from the execution in 3 will be the global file $R5_Global.out$, while the group ones are $R5_G0NP2.out$, $R5_G1NP10.out$.

Both types of files show first the configuration parameters used for the execution, and then the gathered times of the simulation. Global file contains:

- Configuration parameters.
- Times: T_{spawn} , T_{spawn_real} , T_{SR} , T_{AR} and T_{total} .

On the other hand, other files contain the following:

- Configuration parameters.
- Stage parameters.
- Resize parameters of the group Y .
- Measured times T_{iter} and T_{stages} .
- Total iterations executed while an asynchronous resize was being performed ($Asynciters$).

Examples of output for these intermediate files can be found at section ??.

5 Analysis

The results obtained in the simulation during the execution of a malleable application have been stored in the output files described below. These information can be studied with detail in order to analyze its behaviour and performances. The tool provides a script that from these files generates others and graphics that that helps to carry out the study. This analysis is detailed below.

5.1 Analysis script

The *analyser.sh* script (Listing 7) reads the intermediate output files and creates various data files and graphs that display the information in a more direct and useful way. The script expects the following arguments:

1. *InputDirectory*: Directory path where the intermediate files from the simulations are stored. It is mandatory.

```
bash analyser.sh InputDirectory OutputDirectory CreateGraphics
```

Listing 7: Script *analyser.sh* example of use.

2. *OutputDirectory*: Directory path where the analysis results should be saved. Optional with default to current directory.
3. *CreateGraphics* Integer value to indicate whether to generate graphics (1) or not (0). Optional with default value to 1.

5.2 Analysis files

As a result of executing the *analyser.sh* script, the following files are created:

- Three files in CSV format to be used as data sets: *data_global.csv*, *data_malleability.csv* and *data_iterations.csv*.
- A set of images where each one shows:
 1. A color-map showing for a malleable application simulation and for each combination parent/child which is the better spawn method and strategy for expanding/shrinking. In Figure 2 is shown an example where the initial and final processes are 1, 10, 20, 40, 80 and 120. The parameter *NP* is the number of parents while *NC* are children and the number inside each cell shows the method that has provided the best performance in resizing.
 2. A color-map showing for a malleable application simulation and for each combination parent/child which is the better redistribution method.
 3. A set of x images, where x is the number of parent processes. Each image is a graphic bar shows $y+1$ bars, where y is the different number of child processes. The additional bar represents the execution time without malleability, while the others represents execution time when malleability is applied to resize to another number of processes. Also, each bar is divided in time executed by parents (Blue), time used for resizing (Orange) and time used by children (Green). In Figure 3 is shown an example for 10 initial processes, resizing to 1, 20, 40, 80 or 120 processes.

The CSV files contain information about a simulation which may consist of multiple malleable application executions each with different configurations. These CSV files format are described at appendix A. These files are:

- *data_global.csv*: Contains complete information about all the executions performed and their resizing
- *data_malleability.csv*: Contains detailed data of each resizing carried out in each execution.

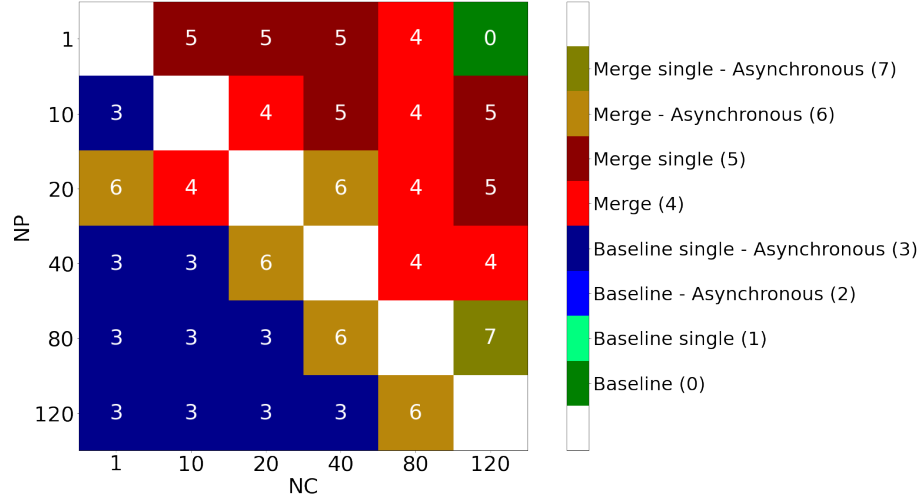


Figure 2: Color-map of preferred methods depending on the number of initial processes(NP) and final processes(NC) in an application.

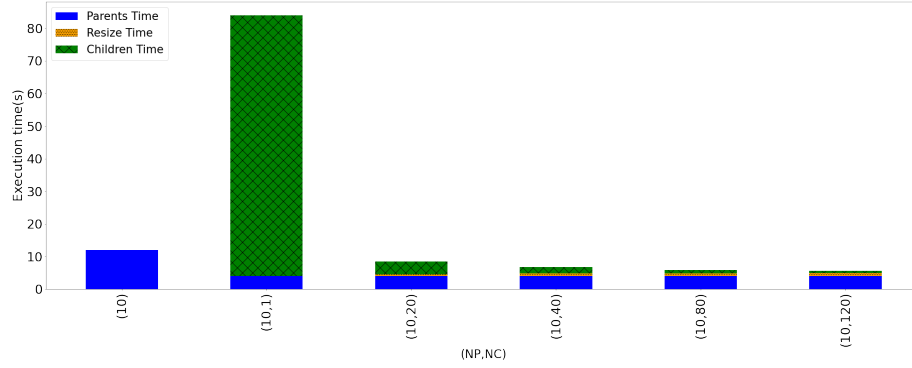


Figure 3: Bar plot of execution times depending on the initial and final processes respectively (NP, NC) for a given application. NP is fixed to 10 and NC varies depending on the executed simulations. For each bar, the time is divided in time executed by parents (Blue), time used for resizing (Orange) and time used by children (Green)

- `data_iterations.csv`: Contains detailed data about each performed iteration in each execution. Simplification of the previous two data sets to access the iterations and study how they are affected by asynchronous resizes.

```
bash singleRunCustom.sh configuration.ini
```

Listing 8: Script *singleRunCustom.sh* example of use.

```
bash singleRunCustom.sh configuration.ini 1 10 $HOME/myDir
```

Listing 9: Script *singleRunCustom.sh* example of use with directory path and number of executions.

6 Examples of use

In this section various examples of use for the synthetic application are provided.

6.1 Single executions

Single runs are called differently depending on whether we want to use a resource manager or not. First you will see the ones that do not use a resource manager, and then the ones that do.

Without RMS Executions: To perform a simple execution without a Resource Manager the script *singleRunCustom.sh* is called with the **bash** command, which is located in the *Exec* directory. To test if the simulation works call it using Listing 8. The result will be saved by default at the directory *Results* inside the malleability benchmark path.

The file *configuration.ini* can be found inside the directory *Exec* and can be used as a baseline for the user to create their own configuration files. This configuration file simulates a Conjugate Gradient application (Described at Listing 13) with the sparse matrix *audikw_1* for a maximum of 100 iterations. It starts with 2 processes and resizes to 10 processes.

If the user wants their results to be saved at *\$HOME/myDir* and execute it 10 times, they should follow the example at Listing 9. The one in the example indicates the results file identifier. The results are shown below, where the identifier is highlighted in red and bold:

- **R1**.Global – Global file
- **R1**.G0NP0 – Resize file for group 0
- **R1**.G1NP0 – Resize file for group 1

With RMS Executions: If the user wants to perform their own allocation with a RMS, is possible to do so with the script *singleRunCustom.sh*. It should be called with **srun** or **sbatch** with the desired options. An example is shown at Listing 10 with the command **sbatch** and only the mandatory parameter, the configuration file. On the other hand, Listing 11 shows an example for the command **srun**, saving the results at *\$HOME/myDir* and executing the simulation 10 times.

```
sbatch -N 2 singleRunCostum.sh configuration.ini
```

Listing 10: Script *singleRunCostum.sh* example of use with **sbatch**.

```
srun -N 2 singleRunCostum.sh configuration.ini 0 10 $HOME/myDir
```

Listing 11: Script *singleRunCostum.sh* example of use with **srun** and selecting an output directory for 10 executions.

Nevertheless, if the user wants to use a RMS but do not want to take care of the resource allocation, is possible to use the script *singleRun.sh*. This script takes care of the resource allocation and must be called with the **bash** command. It is located in the *Exec* directory. To perform a simulation with automatic resource allocation call it using Listing 12. The result will be saved by default at the directory *Results* inside the malleability benchmark path.

Both scripts have the same parameters, the only difference is in the resource allocation.

6.2 Complex execution with Slurm

Using the script *multipleRuns.sh* is possible to indicate a more complex configuration file and the script takes cares of creating and executing all possible combinations in it. This more complex file is described in Section 4.2. To use it is mandatory to have installed Slurm.

In the directory *Exec* can be seen the file *configuration_array.ini* as an example and to use it along the script *multipleRuns.sh*. In Listing 14 is shown an example of how to use this script. The results will be saved in a new directory inside the *Results* directory.

If the complex configuration file used was the one described at Listing 5, a total of 24 configurations would have been created and executed. As for each array, all elements are used to create each possible combination of them. Also, from Listings 15 to 38 are shown each individual configuration for those values that change in each configuration file. In each listing is highlighted in red what changes from the previous file.

```
bash singleRun.sh configuration.ini
```

Listing 12: Script *singleRun.sh* example of use.

```

[general]
Total_Stages=4
Granularity=100000
Total_Resizes=1
SDR=0
ADR=0
;end [general]
[stage0]
Stage_Type=0
Stage_Bytes=0
Stage_Time=0.17
;end [stage0]
[stage1]
Stage_Type=6
Stage_Bytes=8
Stage_Time=0
;end [stage1]
[stage2]
Stage_Type=6
Stage_Bytes=8
Stage_Time=0
;end [stage2]
[stage3]
Stage_Type=4
Stage_Bytes=7549560
Stage_Time=0
;end [stage3]
[resize0]
Iters=90
Procs=2
FactorS=0.5
Dist=spread
Spawn_Method=0
Spawn_Strategy=0
Asynch_Redistribution_Type=0
;end [resize0]
[resize1]
Iters=10
Procs=10
FactorS=0.1
Dist=spread
Spawn_Method=0
Spawn_Strategy=0
Asynch_Redistribution_Type=0
;end [resize1]

# Number of stages
# Computation stages problem size
# Number of resizes
# Total redistributed bytes
# Total asynchronous redistributed bytes

# Computation stage
# Stage type. Perform pi computation
# Bytes to communicate
# Stage duration , 170ms

# Communication stage
# Perform Allreduce
# with 8 bytes
# not used

# Communication stage
# Perform Allreduce
# with 8 bytes
# not used

# Communication stage
# Perform Allgather
# with ~7.5Mb
# not used

# Total iterations for this group
# Number of processes in this group
# Scalability factor
# Physical process mapping type
# Spawn method
# Spawn strategy
# Type of asynchronous redistribution

# Total iterations for this group
# Number of processes in this group
# Scalability factor
# Physical process mapping type
# Spawn method
# Spawn strategy
# Type of asynchronous redistribution

```

Listing 13: CG application as a configuration file.

```
bash multipleRuns.sh configuration_array.ini
```

Listing 14: Script *multipleRuns.sh* example of use.

```

Configuration file 0
[general]
...
ADR=0
.....
[resize0]
...
Procs=2
FactorS=0.5
Dist=spread
...
;end [resize0]
[resize1]
..
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 15: Resulted configuration file 0 from a complex configuration file.

```

Configuration file 1
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=2
FactorS=0.5
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 16: Resulted configuration file 1 from a complex configuration file.

```

Configuration file 2
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=2
FactorS=0.5
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 17: Resulted configuration file 2 from a complex configuration file.

```

Configuration file 3
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=2
FactorS=0.5
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 18: Resulted configuration file 3 from a complex configuration file.

```

Configuration file 4
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 19: Resulted configuration file 4 from a complex configuration file.

```

Configuration file 5
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 20: Resulted configuration file 5 from a complex configuration file.

```

Configuration file 6
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 21: Resulted configuration file 6 from a complex configuration file.

```

Configuration file 7
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 22: Resulted configuration file 7 from a complex configuration file.

```

Configuration file 8
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 23: Resulted configuration file 8 from a complex configuration file.

```

Configuration file 9
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 24: Resulted configuration file 9 from a complex configuration file.

```

Configuration file 10
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 25: Resulted configuration file 10 from a complex configuration file.

```

Configuration file 11
[general]
...
ADR=0
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 26: Resulted configuration file 11 from a complex configuration file.

```

Configuration file 12
[general]
...
Asynch-Percentage-Redistribution=100
.....
[resize0]
...
Procs=2
FactorS=0.5
Dist=spread
...
;end [resize0]
[resize1]
..
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 27: Resulted configuration file 12 from a complex configuration file.

```

Configuration file 1
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=2
FactorS=0.5
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 28: Resulted configuration file 13 from a complex configuration file.

```

Configuration file 14
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=2
FactorS=0.5
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 29: Resulted configuration file 14 from a complex configuration file.

```

Configuration file 15
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=2
FactorS=0.5
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 30: Resulted configuration file 15 from a complex configuration file.

```

Configuration file 16
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 31: Resulted configuration file 16 from a complex configuration file.

```

Configuration file 17
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 32: Resulted configuration file 17 from a complex configuration file.

```

Configuration file 18
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 33: Resulted configuration file 18 from a complex configuration file.

```

Configuration file 19
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=4
FactorS=0.3
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 34: Resulted configuration file 19 from a complex configuration file.

```

Configuration file 20
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 35: Resulted configuration file 20 from a complex configuration file.

```

Configuration file 21
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=spread
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 36: Resulted configuration file 21 from a complex configuration file.

```

Configuration file 22
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=10
FactorS=0.18
...
;end [resize1]

```

Listing 37: Resulted configuration file 22 from a complex configuration file.

```

Configuration file 23
[general]
...
ADR=100
;end [general]
.....
[resize0]
..
Procs=8
FactorS=0.2
Dist=compact
...
;end [resize0]
[resize1]
...
Procs=20
FactorS=0.1
...
;end [resize1]

```

Listing 38: Resulted configuration file 23 from a complex configuration file.

```
bash multipleRuns.sh configuration_array.ini 10 $HOME/myDir raw
```

Listing 39: Script *multipleRuns.sh* example of use for 10 repetitions and results saved at *\$HOME/myDir/raw*.

```
bash CheckRun.sh $HOME/myDir
```

Listing 40: Script *CheckRun.sh* example of use when the results are at *\$HOME/myDir*.

In Listing 39 is shown another example of a complex configuration file, where 10 repetitions for each configuration are requested. Also, the results should be saved inside *\$HOME/myDir* at a new directory of name *raw*.

6.3 Error checking examples

After performing a simulation is recommended to check if the executions have been executed properly. Using the script *CheckRun.sh* at directory *Exec* in the malleability benchmark path.

An example can be seen at Listing 40 where the results are located at *\$HOME/myDir*. If there is any error, those runs will be executed again but if there is a fatal error, the script will notify it.

6.4 Analysis script execution examples

To obtain legible results the script *analyser.sh* at directory *Analysis* in the malleability benchmark path. An example of usage can be seen at Listing 41, where the results are at *\$HOME/myDir* and should be saved to *otherDir* inside current directory.

At directory *otherDir* two text files, three CSV files and a number of images will be created. The number of images depends on the simulations used in the analyser.

Another example is shown at Listing 42, where the results are at *path/to/malleability_benchmark/Results* and the outcomes are saved in the current directory (*\$HOME/Documents*).

In both examples, at directory *otherDir*(First examples) and the current directory(Second example) two text files, three CSV files and a number of images will be created. The number of images depends on the simulations used in the analyser.

```
bash analyser.sh $HOME/myDir otherDir
```

Listing 41: Script *analyser.sh* example of use when the results are at *\$HOME/myDir* and stored at *otherDir*.

```
bash analyser.sh path/to/malleability_benchmark/Results
```

Listing 42: Script *analyser.sh* example of use when the results are at path/to/-malleability_benchmark/Results and stored at current directory.

A Output CSV files format

In this appendix is described the format of each final output file, in order to allow users to analyze by themselves the obtained results.

Each of the rows of file *data_global.csv* represents a complete execution of a malleable application. Columns can be grouped according to their meaning. On the one hand, there are those showing information on the resizing, on the computation in each iteration and the results obtained.

- Columns related to resize configuration:
 - Total_Resizes: It is an integer that indicates number of performed resizes (**Total_Resizes** in configuration file).
 - Total_Groups: It is an integer indicating the number of processes groups for the malleable execution. It is equal to **Total_Resizes**+1. Each of the following five columns is a list of data showing the concrete information for each group in a resize. They are a ordered array of integers with size *Total_Groups*.
 - Groups: Number of processes in a group.
 - Dist: Physical distribution of processes in a group. Its possible values are Compact(0) and Spread(1).
 - Spawn_Method: Spawn method for a resize. Its possible values are Baseline(0) and Merge(1).
 - Spawn_Strategy: Spawn strategy for a resize. Its possible values are None(0), Single(1), Asynchronous(2) and All(3).
 - Asynch_Redistribution_Type: Redistribution asynchronous method for each resize. Array of integers with size *Total_Resizes*. Its possible values are Point-to-point(0), Alltoall parents(1), Alltoall children(2), Threaded(3).
 - SDR: Number of bytes redistributed synchronously in each redistribution stage. Integer.
 - ADR: Number of bytes redistributed asynchronously in each redistribution stage. Integer.
 - DR: Number of bytes redistributed in each redistribution stage. Is equal to *SDR* + *ADR*. Integer.
- Columns related to application type:

- Granularity: It is an integer that indicates problem size for computation stages.
- FactorS: Speedup factor to emulate how the performance of computation stages are affected by the number of processes. Ordered array of Integers of size *Total_Groups*.
- Total_Stages: It is an integer that represents number of stages in an iteration. The three following columns contain concrete information about each of stages. They will be an ordered array of size *Total_Stages*.
- Stage_Types: Each element indicates stage type (computation/com-
munication). Its possible values are PI_Computation(0), Matrix_Computation(1),
Point_communication(2), Bcast_communication(3), Allgatherv_communication(4),
Reduce_communication(5) and Allreduce_communication(6).
- Stage_Times: Each element represents the stage time.
- Stage_Bytes: Each element indicates bytes communicated in this
stage. For computation stages is always 0.
- *Latency_{net}*: Latency of the used network.
- *Bw_{net}*: Bandwidth of the used network.
- *Latency_{shm}*: Latency of the used shared memory.
- *Bw_{shm}*: Bandwidth of the used shared memory.

- Results columns:

- Iters: It is an ordered array of integers with size *Total_Groups* that
contains executed iterations by each group.
- Asynch_Iters: It is an ordered array of integers with size *Total_Resizes*
that represents the total iterations executed by the parents while an
asynchronous resize was taking place.
- *T_{iter}*: Array of arrays (2 levels) which contains the times of each
iteration. The upper array refers to the iterations of each group
and has a size of *Total_Groups*. The inner array contains floats,
and the size of the array with index *i* is *Iters[i]*, where *i* refers to
a group of processes. Therefore, to check the iteration time of the
last iteration in the first group of processes it will be accessed as
T_{iter}[0][*Iters*[0] – 1].
- *T_{stages}*: Array of 3 levels which contains the times of each performed
stage in the execution. The first level contains arrays of groups and is
of size *Total_Groups*. The second level contains an array of iterations,
and the size of the array with index *i* is *Iters[i]*, where *i* refers to a
group of processes. Lastly, the third level stores for a specific iteration,
an array of floats values where each one represents the time of a
stage in the iteration. Has a size of *Total_Stages*. Therefore, to check
the stage time of the last stage in the last iteration of the first group of
processes it will be accessed as *T_{iter}*[0][*Iters*[0] – 1][*Total_Stages* – 1].

- T_{spawn} : Ordered array of spawn times for each resize. Refers to the time T_{spawn} in the monitoring module (Subsection 2.4). Array of floats with size $Total_Resizes$.
- T_{spawn_real} : Ordered array of real spawn times for each resize. Refers to the time T_{spawn_real} in the monitoring module (Subsection 2.4). Array of floats with size $Total_Resizes$.
- T_{SR} : Ordered array of synchronous data redistribution times for each resize. Refers to the time T_{SR} in the monitoring module (Subsection 2.4). Array of floats with size $Total_Resizes$.
- T_{AR} : Ordered array of asynchronous data redistributions times for each resize. Refers to the time T_{AR} in the monitoring module (Subsection 2.4). Array of floats with size $Total_Resizes$.
- T_{total} : Total execution time of the application. Float value.

The other two files generated by *analyser.sh* script, *data_malleability.csv* and *data_iterations.csv*, are focused on helping the analysis of behaviour of malleable application while resizing is being performed above when it is carried out asynchronously. In both files, each row represents a different malleable running and its columns are a subset of those described in the previous file.

The second data set *data_malleability.csv*, is intended to facilitate the analysis of different configurations in malleability only, where all the execution data is not needed. Each row represents a single resize. This data set has the next columns:

- Resize configuration columns:
 - NP: Number of parent processes before the resize. Integer.
 - NC: Number of children processes after the resize. Integer.
 - Dist: Physical distribution of processes for the parents and children. Array of two integers. Its possible values are Compact(0) and Spread(1).
 - Spawn.Method: Spawn method used for the resize. Integer. Its possible values are Baseline(0) and Merge(1).
 - Spawn.Strategy: Spawn strategy used for the resize. Integer. Its possible values are None(0), Asynchronous(1), Single(2), All(3).
 - Asynch.Redistribution.Type: Data redistribution asynchronous method for the resize. Integer. Its possible values are Point-to-point(0), Alltoall parents (1), Alltoall children(2), Threaded(3).
 - SDR: Number of bytes redistributed synchronously in the redistribution stage. Integer.
 - ADR: Number of bytes redistributed asynchronously in the redistribution stage. Integer.
 - DR: Number of bytes redistributed in the redistribution stage. Is equal to $SDR + ADR$. Integer.

- Compute configuration columns:
 - Total_Stages: Number of stages. Integer.
 - Granularity: Problem size for computation stages. Integer.
 - FactorS: Speedup factor to emulate how the performance of computation stages are affected by the number of processes at column *NP*. Integer.
 - Stage_Type: Ordered array of stages types for each stage of the iterations. Array of integer with size *Total_Stages*. Its possible values are PI_Computation(0), Matrix_Computation(1), Point_communication(2), Bcast_communication(3), Allgatherv_communication(4), Reduce_communication(5) and Allreduce_communication(6).
 - Stage_Time: Ordered array of stage times for each stage of the iterations. Array of integer with size *Total_Stages*.
 - Stage_Bytes: Ordered array of bytes communicated for each stage of the iterations. Array of integer with size *Total_Stages*.
 - *Latency_{net}*: Latency of the used network.
 - *Bw_{net}*: Bandwidth of the used network.
 - *Latency_{shm}*: Latency of the used shared memory.
 - *Bw_{shm}*: Bandwidth of the used shared memory.
- Results columns:
 - Iters: Quantity of performed iterations by the parents. Is the sum of *Asynch_Iters* and synchronous iterations. Integer.
 - Asynch_Iters: Quantity of performed iterations by the parents while an asynchronous resize was taking place. Integer.
 - *T_{iter}*: Array of times for each performed iteration in this group. The array contains floats and has a size of *Iters* elements. Therefore, to check the iteration time of the last iteration it will be accessed as *T_{iter}*[*Iters* – 1].
 - *T_{stages}*: Array of 2 levels which contains the times of each performed stage by the group. The first level contains an array of iterations, with an array size of *Iters*. On the other hand, the second level stores for a specific iteration, an array of floats values where each one represents the time of a stage in the iteration. Has a size of *Total_Stages*. Therefore, to check the stage time of the last stage in the last iteration it will be accessed as *T_{stages}*[*Iters* – 1][*Total_Stages* – 1].
 - *T_{spawn}*: Spawn times for the resize. Refers to the time *T_{spawn}* in the monitoring module (Subsection 2.4). Float value.
 - *T_{spawn_real}*: Real spawn times for the resize. Refers to the time *T_{spawn_real}* in the monitoring module (Subsection 2.4). Float value.

- T_{SR} : Synchronous data redistribution time for the resize. Refers to the time T_{SR} in the monitoring module (Subsection 2.4). Float value.
- T_{AR} : Asynchronous data redistribution time for the resize. Refers to the time T_{AR} in the monitoring module (Subsection 2.4). Float value.

The final data set *data_iterations.csv* is a simplification of the previous data sets to focus an analysis on how each iteration has been performed. Each row represents a single iteration. This data set has the next columns:

- NP: Number of processes. Integer.
- The next values should be ignored if *Asynch_Iters* is 0 (*Dist[0]* is an exception). Resize configuration columns:
 - NC: Number of children processes after the resize. Integer.
 - Dist: Physical distribution of processes for the parents and children. Array of two integers. Its possible values are Compact(0) and Spread(1).
 - Spawn_Method: Spawn method used for the resize. Integer. Its possible values are Baseline(0) and Merge(1).
 - Spawn_Strategies: Spawn strategy used for the resize. Integer. Its possible values are None(0), Asynchronous(1), Single(2), All(3).
 - Asynch_Redistribution_Type: Data redistribution asynchronous method for the resize. Integer. Its possible values are Point-to-point(0), Alltoall parents (1), Alltoall children (2), Threaded(3).
 - SDR: Number of bytes redistributed synchronously in the redistribution stage. Integer.
 - ADR: Number of bytes redistributed asynchronously in the redistribution stage. Integer.
 - DR: Number of bytes redistributed in the redistribution stage. Is equal to $SDR + ADR$. Integer.
- Compute configuration columns:
 - Total_Stages: Number of stages. Integer.
 - Granularity: Problem size for computation stages. Integer.
 - FactorS: Speedup factor to emulate how the performance of computation stages are affected by the number of processes at column *NP*. Integer.
 - Stages_Type: Ordered array of stage types for each stage of the iterations. Array of integer with size *Total_Stages*. Its possible values are PI.Computation(0), Matrix.Computation(1), Point_communication(2), Bcast_communication(3), Allgatherv_communication(4), Reduce_communication(5) and Allreduce_communication(6).

- Stage_Time: Ordered array of stage times for each stage of the iterations. Array of integer with size *Total_Stages*.
- Stage_Bytes: Ordered array of bytes communicated for each stage of the iterations. Array of integer with size *Total_Stages*.
- *Latency_{net}*: Latency of the used network.
- *Bw_{net}*: Bandwidth of the used network.
- *Latency_{shm}*: Latency of the shared memory.
- *Bw_{shm}*: Bandwidth of the used shared memory.
- Results columns:
 - *Asynch_Iters*: Indicates whether this iteration has been performed normally (0) or during an asynchronous resize (1). Integer.
 - *T_{iter}*: Iteration time. Float value
 - *T_{stage}*: Array of each stage time in the iteration. Array of floats with size *Total_Stages*. Therefore, to check the stage time of the last stage in the iteration it will be accessed as $T_{stages}[Total_Stages - 1]$.

References

- [1] I. Comprés, A. Mo-Hellenbrand, M. Gerndt, and H. J. Bungartz, “Infrastructure and API Extensions for Elastic Execution of MPI Applications,” in *ACM International Conference Proceeding Series*, vol. 25-28-Sept. New York, New York, USA: ACM Press, 2016, pp. 82–97.
- [2] S. Iserte, H. Martínez, S. Barrachina, M. Castillo, R. Mayo, and A. J. Peña, “Dynamic Reconfiguration of Noniterative Scientific Applications,” *The International Journal of High Performance Computing Applications*, p. 109434201880234, sep 2018.
- [3] R. Sudarsan, C. J. Ribbens, and D. Farkas, “Dynamic Resizing of Parallel Scientific Simulations: A Case Study Using LAMMPS,” in *Computational Science – ICCS*, vol. 5544, 2009, pp. 175–184.
- [4] S. Iserte and K. Rojek, “An Study of the Effect of Process Malleability in the Energy Efficiency on GPU-based Clusters,” *The Journal of Supercomputing*, pp. 1–20, oct 2019. [Online]. Available: <http://link.springer.com/10.1007/s11227-019-03034-x>
- [5] S. Prabhakaran, M. Neumann, S. Rinke, F. Wolf, A. Gupta, and L. V. Kale, “A Batch System with Efficient Adaptive Scheduling for Malleable and Evolving Applications,” in *2015 IEEE International Parallel and Distributed Processing Symposium*, 2015, pp. 429–438.